

NODE.JS

Integración de JavaScript moderno en entorno de ejecución backend

1. Objetivos de la sesión

- Comprender qué es Node.js y cómo funciona internamente
- Ejecutar código JavaScript en entorno servidor
- Aplicar conceptos de ES6+ en Node.js
- Trabajar con módulos reales (`import/export`, `require`)
- Entender la asincronía en contexto backend
- Gestionar dependencias mediante npm

2. ¿Qué es Node.js?

Node.js es un **entorno de ejecución de JavaScript basado en el motor V8**, diseñado para ejecutar código fuera del navegador.

A diferencia del entorno web, Node.js:

- No dispone de DOM ni APIs del navegador
- Proporciona acceso directo al sistema de ficheros
- Permite crear servidores y aplicaciones backend

Arquitectura

Node.js sigue un modelo:

- Event-driven
- Non-blocking I/O
- Single-threaded (con event loop)

3. Modelo de ejecución en Node.js

El núcleo de Node.js se basa en:

- **Call Stack**
- **Event Loop**
- **Libuv (gestión de I/O asíncrona)**

Ejemplo técnico

```
const fs = require("fs");

console.log("Inicio");

fs.readFile("archivo.txt", "utf8", (err, data) => {
  if (err) throw err;
  console.log("Contenido:", data);
});

console.log("Fin");
``
```

Resultado:

```
Inicio
Fin
Contenido: ...
```

Interpretación:

- La lectura de archivo es no bloqueante
- El callback se ejecuta cuando el sistema lo permite
- Se aplica el mismo modelo que ya conocían en el navegador

4. Instalación y entorno de trabajo

Instalación

- Descargar Node.js (LTS)
- Verificar instalación:

node -v

npm -v

Uso en VS Code

Estructura mínima de proyecto:

```
/mi-proyecto
├── app.js
└── package.json
```

Inicialización:

npm init -y

Ejecución:

node app.js

5. Sistema de módulos en Node.js

5.1 CommonJS (tradicional Node.js)

```
// utils.js
function sumar(a, b) {
  return a + b;
}

module.exports = { sumar };

// app.js
const { sumar } = require("./utils");

console.log(sumar(2, 3));
```

5.2 ES Modules (moderno)

Activación en package.json:

```
{
  "type": "module"
}
```

Uso:

```
// utils.js
export const sumar = (a, b) => a + b;

// app.js
import { sumar } from "./utils.js";
```

Node.js soporta ambos sistemas, pero ES Modules es el estándar actual.

6. Uso de JavaScript moderno en Node.js

6.1 Arrow functions

```
const procesarDatos = (datos) => datos.map(d => d * 2);
```

Uso típico en callbacks y transformaciones de datos.

6.2 Destructuring en Node

Ejemplo con módulos internos:

```
const { readFile } = require("fs");
```

```
readFile("file.txt", "utf8", (err, data) => {  
  console.log(data);  
});  
..
```

6.3 Template strings en logs

```
const puerto = 3000;  
console.log(`Servidor escuchando en puerto ${puerto}`);
```

7. Closures en Node.js

Uso habitual en:

- Middleware
- Configuración
- Encapsulación de lógica

Ejemplo práctico

```
function crearLogger(prefijo) {  
  return function(mensaje) {  
    console.log([`${prefijo}`] `${mensaje}`);  
  };  
}
```

```
const logger = crearLogger("APP");
```

```
logger("Servidor iniciado");  
``
```

Esto permite crear sistemas configurables sin exponer estado global.

8. Programación funcional en el backend

Uso intensivo en procesos de datos.

Ejemplo

```
const datos = [  
  { usuario: "Ana", activo: true },  
  { usuario: "Luis", activo: false }  
];
```

```
const activos = datos  
  .filter(u => u.activo)  
  .map(u => u.usuario);
```

```
console.log(activos);
```

Aplicación directa en:

- APIs REST
- Servicios de procesamiento de datos
- Transformación de respuestas

9. Manejo de arrays y objetos en Node

Caso práctico: procesamiento de JSON

```
const fs = require("fs");

fs.readFile("datos.json", "utf8", (err, data) => {
  const usuarios = JSON.parse(data);

  const mayores = usuarios.filter(u => u.edad > 18);

  console.log(mayores);
});
```

Este patrón es muy habitual en backend.

10. Gestión de dependencias con npm

Concepto

npm es el gestor de paquetes de Node.js.

Instalación de paquetes

```
npm install axios
```

Uso

```
const axios = require("axios");

axios.get("https://api.example.com")
  .then(res => console.log(res.data));
```

package.json

Define:

- Dependencias
- Scripts
- Configuración del proyecto

11. Scripts y automatización

```
{  
  "scripts": {  
    "start": "node app.js",  
    "dev": "nodemon app.js"  
  }  
}
```

Ejecución:

npm run dev

12. Caso práctico

Objetivo: crear un procesador de usuarios

```
// usuarios.js  
export const procesarUsuarios = (usuarios) => {  
  return usuarios  
    .filter(u => u.activo)  
    .map(u => ({  
      nombre: u.nombre,  
      email: u.email  
    }));  
};  
  
// app.js  
import { procesarUsuarios } from "./usuarios.js";  
  
const usuarios = [  
  { nombre: "Ana", email: "ana@test.com", activo: true },  
  { nombre: "Luis", email: "luis@test.com", activo: false }  
];  
  
const resultado = procesarUsuarios(usuarios);  
  
console.log(resultado);
```

Relación con Ionic

Este bloque es fundamental porque:

- Ionic consume APIs creadas en Node.js
- Se comparte lógica JavaScript entre frontend y backend
- Se trabaja con el mismo modelo de asincronía

Async/Await en Node.js

1. Fundamento conceptual

`async/await` es una **abstracción sobre Promises** introducida en ES2017 que permite escribir código asíncrono con una sintaxis más cercana al flujo síncrono, sin bloquear el event loop.

En Node.js, donde el modelo de ejecución es **no bloqueante y basado en eventos**, `async/await` simplifica enormemente:

- Lectura/escritura de archivos
- Llamadas a APIs
- Acceso a bases de datos
- Operaciones concurrentes

2. Relación con Promises

Para entender `async/await`, es imprescindible comprender que:

- `async` convierte una función en una función que devuelve siempre una Promise
- `await` solo puede usarse dentro de funciones `async`
- `await` pausa la ejecución de la función hasta que la Promise se resuelve

Ejemplo base con Promise

```
const fs = require("fs").promises;

function leerArchivo() {
  return fs.readFile("datos.txt", "utf8");
}

leerArchivo()
  .then(data => console.log(data))
  .catch(err => console.error(err));
```

Equivalente con `async/await`

```
const fs = require("fs").promises;

async function leerArchivo() {
  try {
    const data = await fs.readFile("datos.txt", "utf8");
    console.log(data);
  } catch (error) {
    console.error(error);
  }
}
```

3. Comportamiento interno

Cuando el motor encuentra `await`:

1. Evalúa la Promise
2. Suspende la ejecución de la función (no bloquea el hilo principal)
3. Libera el control al event loop
4. Reanuda la ejecución cuando la Promise se resuelve

Esto es clave: **no bloquea Node.js**, solo pausa esa función.

4. Uso en Node.js

4.1 Lectura de archivos

```
import { readFile } from "fs/promises";

async function cargarConfig() {
  const contenido = await readFile("./config.json", "utf8");
  return JSON.parse(contenido);
}

cargarConfig().then(config => console.log(config));
```

4.2 Consumo de APIs

```
import fetch from "node-fetch";

async function obtenerUsuarios() {
  const response = await fetch("https://jsonplaceholder.typicode.com/users");
  const data = await response.json();

  return data;
}
```

4.3 Funciones encadenadas

```
async function proceso() {
  const datos = await obtenerUsuarios();
  const filtrados = datos.filter(u => u.id > 5);
  return filtrados;
}
```

5. Manejo de errores

Uno de los puntos clave de `async/await` es la gestión de errores mediante `try/catch`.

Ejemplo

```
async function ejemploError() {  
  try {  
    const data = await Promise.reject("Error simulado");  
  } catch (error) {  
    console.error("Capturado:", error);  
  }  
}
```

Error sin capturar

```
async function fallo() {  
  const data = await Promise.reject("Error grave");  
}
```

```
fallo(); // UnhandledPromiseRejection
```

En Node.js, esto puede provocar que la aplicación falle si no se gestiona correctamente.

6. Ejecución concurrente vs secuencial

Un error común es usar `await` de forma secuencial cuando podría ser concurrente.

Ejecución secuencial (ineficiente)

```
async function secuencial() {  
  const a = await tareaA();  
  const b = await tareaB();  
}
```

Ejecución concurrente (óptima)

```
async function paralelo() {  
  const [a, b] = await Promise.all([tareaA(), tareaB()]);  
}
```

7. Caso práctico

Escenario: cargar datos de usuarios y pedidos

```
async function obtenerUsuarios() {  
  return new Promise(resolve =>  
    setTimeout(() => resolve(["Ana", "Luis"]), 1000)  
  );  
}
```

```
async function obtenerPedidos() {  
  return new Promise(resolve =>  
    setTimeout(() => resolve([100, 200]), 1000)  
  );  
}
```

Versión ineficiente

```
async function cargaSecuencial() {  
  const usuarios = await obtenerUsuarios();  
  const pedidos = await obtenerPedidos();  
  
  console.log(usuarios, pedidos);  
}
```

Tiempo total: ~2 segundos

Versión optimizada

```
async function cargaParalela() {  
  const [usuarios, pedidos] = await Promise.all([  
    obtenerUsuarios(),  
    obtenerPedidos()  
  ]);  
  
  console.log(usuarios, pedidos);  
}
```

Tiempo total: ~1 segundo

8. Integración con módulos modernos

```
// servicio.js
export const obtenerDatos = async () => {
  return
}

// app.js
import { obtenerDatos } from "./servicio.js";

(async () => {
  const datos = await obtenerDatos();
  console.log(datos);
})();
```

Se usa una **IIFE async** para poder usar **await** en el nivel superior.

API REST en Node.js con async/await

1. Arquitectura

Se construirá una API simple de usuarios con:

- Node.js + Express
- Uso de `async/await`
- Separación en capas:
 - rutas
 - controladores
 - servicios

2. Instalación del entorno

```
mkdir api-node
```

```
cd api-node
```

```
npm init -y
```

```
npm install express
```

3. Estructura del proyecto

```
/api-node
├── app.js
├── routes/
│   └── users.routes.js
├── controllers/
│   └── users.controller.js
├── services/
│   └── users.service.js
```

4. Simulación de base de datos (servicio)

Archivo: `services/users.service.js`

```
let usuarios = [
  { id: 1, nombre: "Ana", email: "ana@test.com" },
  { id: 2, nombre: "Luis", email: "luis@test.com" }
];

export const getUsuarios = async () => {
  return usuarios;
};

export const getUsuarioById = async (id) => {
  const usuario = usuarios.find(u => u.id === id);

  if (!usuario) {
    throw new Error("Usuario no encontrado");
  }

  return usuario;
};

export const createUsuario = async (data) => {
  const nuevo = {
    id: Date.now(),
    ...data
  };

  usuarios.push(nuevo);
  return nuevo;
};
..
```

5. Controlador (uso de async/await)

Archivo: `controllers/users.controller.js`

```
import * as userService from "../services/users.service.js";

export const obtenerUsuarios = async (req, res) => {
  try {
    const usuarios = await userService.getUsuarios();
    res.json(usuarios);
  } catch (error) {
    res.status(500).json({ mensaje: error.message });
  }
};

export const obtenerUsuario = async (req, res) => {
  try {
    const id = parseInt(req.params.id);
    const usuario = await userService.getUsuarioById(id);

    res.json(usuario);
  } catch (error) {
    res.status(404).json({ mensaje: error.message });
  }
};

export const crearUsuario = async (req, res) => {
  try {
    const { nombre, email } = req.body;

    const nuevo = await userService.createUsuario({ nombre, email });

    res.status(201).json(nuevo);
  } catch (error) {
    res.status(500).json({ mensaje: error.message });
  }
};
```

6. Rutas

Archivo: `routes/users.routes.js`

```
import { Router } from "express";
import {
  obtenerUsuarios,
  obtenerUsuario,
  crearUsuario
} from "../controllers/users.controller.js";

const router = Router();

router.get("/", obtenerUsuarios);
router.get("/:id", obtenerUsuario);
router.post("/", crearUsuario);

export default router;
```

7. Aplicación principal

Archivo: `app.js`

```
import express from "express";
import usersRoutes from "../routes/users.routes.js";

const app = express();

app.use(express.json());
app.use("/api/users", usersRoutes);

const PORT = 3000;

app.listen(PORT, () => {
  console.log(Servidor ejecutándose en puerto ${PORT});
});
```

8. Activar módulos ES

En `package.json`:

```
{
  "type": "module"
}
```

9. Pruebas de la API

Obtener todos los usuarios

GET `http://localhost:3000/api/users`

Obtener usuario por ID

GET `http://localhost:3000/api/users/1`

Crear usuario

POST `http://localhost:3000/api/users`
Content-Type: `application/json`

Body:

```
{  
  "nombre": "Carlos",  
  "email": "carlos@test.com"  
}
```

10. Análisis técnico del uso de async/await

10.1 Flujo en controlador

```
const usuarios = await userService.getUsuarios();
```

- Se suspende la ejecución del controlador
- No se bloquea el event loop
- Permite código secuencial legible

10.2 Manejo de errores estructurado

```
try {  
  ...  
} catch (error) {  
  res.status(500).json({ mensaje: error.message });  
}
```

Ventajas:

- Evita callbacks anidados
- Centraliza la gestión de errores
- Mejora la trazabilidad

10.3 Separación de capas

- **Servicio** → lógica de negocio
- **Controlador** → interacción HTTP
- **Ruta** → definición endpoint

Esto refleja arquitectura real en proyectos Node/Ionic.

11. Mejora: manejo global de errores

Ejemplo de middleware:

```
app.use((err, req, res, next) => {  
  console.error(err.stack);  
  res.status(500).json({ error: "Error interno" });  
});
```

12. Versión con asincronía simulada

Para reforzar conceptos, puedes simular latencia:

```
export const getUsuarios = async () => {  
  return new Promise(resolve => {  
    setTimeout(() => resolve(usuarios), 1000);  
  });  
};
```

Esto permite observar claramente el comportamiento del event loop.

13. Relación directa con Ionic

Este patrón es exactamente el que consumirá una app Ionic:

```
async getUsuarios() {  
  return this.http.get(<a href="http://localhost:3000/api/users">"http://localhost:3000/api/users"</a>).toPromise();  
}
```

El uso de `async/await` permite:

- Control del flujo de datos
- Manejo de estados (loading, error)
- Mayor claridad en componentes

Event Loop en Node.js

1. Modelo de ejecución de JavaScript

JavaScript es un lenguaje **single-threaded**, lo que implica que:

- Solo existe una pila de ejecución (Call Stack)
- Solo se ejecuta una instrucción a la vez

Sin embargo, Node.js permite manejar miles de operaciones concurrentes gracias a:

- Event Loop
- Cola de eventos
- APIs asíncronas (libuv)

2. Componentes internos

Para comprender el Event Loop, hay que entender sus elementos:

2.1 Call Stack

Estructura LIFO donde se ejecutan funciones.

```
function a() {  
  b();  
}
```

```
function b() {  
  console.log("Ejecutando");  
}
```

```
a();
```

Stack:

$a \rightarrow b \rightarrow \text{console.log}$

2.2 Heap

Zona de memoria donde se almacenan objetos dinámicos.

2.3 Web APIs / Node APIs

En Node.js, estas APIs están proporcionadas por:

- **libuv**
- Sistema operativo

Ejemplos:

- File System (fs)
- Timers (setTimeout)
- Networking

2.4 Callback Queue

Cola donde se almacenan funciones que deben ejecutarse cuando el Call Stack esté vacío.

2.5 Microtask Queue

Cola de prioridad alta donde se colocan:

- Promises (`.then`, `.catch`, `.finally`)
- `queueMicrotask`
- `process.nextTick` (prioridad incluso superior en Node)

3. Funcionamiento del Event Loop

El Event Loop es un ciclo continuo que:

1. Verifica si el Call Stack está vacío
2. Procesa microtasks
3. Procesa una tarea de la cola principal
4. Repite

4. Ejecución paso a paso

Ejemplo

```
console.log("Inicio");
```

```
setTimeout(() => {  
  console.log("Timeout");  
}, 0);
```

```
Promise.resolve().then(() => {  
  console.log("Promise");  
});
```

```
console.log("Fin");
```

Resultado

Inicio
Fin
Promise
Timeout

Análisis técnico

1. Ejecuta código síncrono:
 - "Inicio"
 - "Fin"
2. `Promise.then` → Microtask queue
3. `setTimeout` → Callback queue
4. Event Loop:
 - Ejecuta microtasks → "Promise"
 - Ejecuta macrotask → "Timeout"

5. Fases del Event Loop en Node.js

A diferencia del navegador, Node.js tiene fases específicas:

Fases principales

1. Timers

- `setTimeout`
- `setInterval`

2. I/O callbacks

- Callbacks de operaciones I/O

3. Idle/Prepare

4. Poll

- Espera y ejecuta I/O

5. Check

- `setImmediate`

6. Close callbacks

Ejemplo

```
setTimeout(() => console.log("Timeout"), 0);
```

```
setImmediate(() => console.log("Immediate"));
```

Resultado puede variar, pero en muchos casos:

Immediate

Timeout

Dependiendo del contexto de ejecución.

6. Microtasks vs Macrotasks

Microtasks (alta prioridad)

- Promises
- `process.nextTick`

Macrotasks

- `setTimeout`
- `setImmediate`
- I/O

Ejemplo

```
setTimeout(() => console.log("timeout"), 0);
```

```
Promise.resolve().then(() => console.log("promise"));
```

```
process.nextTick(() => console.log("nextTick"));
```

Resultado (Node):

```
nextTick  
promise  
timeout
```

Orden:

1. `process.nextTick`
2. Promises
3. Macrotasks

7. Relación con async/await

`async/await` utiliza Promises internamente, por lo tanto:

- `await` genera microtasks
- Se integra en el Event Loop sin bloquear

Ejemplo

```
async function test() {  
  console.log("Inicio");  
  
  await Promise.resolve();  
  
  console.log("Después await");  
}  
  
test();  
console.log("Fin");
```

Resultado:

Inicio
Fin
Después await

Interpretación:

- `await` mueve la continuación a la microtask queue

8. Event Loop + I/O real (Node.js)

Ejemplo con filesystem

```
const fs = require("fs");

fs.readFile("file.txt", () => {
  console.log("Archivo leído");
});
```

```
console.log("Fin script");
```

Resultado:

```
Fin script
Archivo leído
```

El sistema operativo delega la operación de lectura, y cuando termina:

- Se envía el callback al Event Loop

9. Problemas comunes en producción

9.1 Bloqueo del Event Loop

```
while (true) {}
```

Esto bloquea completamente Node.js.

9.2 Operaciones CPU intensivas

```
function calcular() {  
  let total = 0;  
  for (let i = 0; i < 1e9; i++) {  
    total += i;  
  }  
  return total;  
}  
`
```

Solución:

- Worker threads
- External services

9.3 Abuso de process.nextTick

```
function repetir() {  
  process.nextTick(repetir);  
}
```

```
repetir();
```

Esto puede bloquear el Event Loop al priorizar microtasks infinitamente.

10. Caso práctico

Escenario combinado

```
console.log("Inicio");
```

```
setTimeout(() => console.log("Timeout 1"), 0);
```

```
setImmediate(() => console.log("Immediate"));
```

```
Promise.resolve().then(() => console.log("Promise 1"));
```

```
process.nextTick(() => console.log("NextTick"));
```

```
console.log("Fin");
```

Posible salida

Inicio

Fin

NextTick

Promise 1

Immediate

Timeout 1

Comprender el Event Loop es esencial para:

Backend (Node.js)

- Optimizar rendimiento
- Evitar bloqueos
- Diseñar APIs escalables

Frontend / Ionic

- Control de rendering
- Manejo de eventos
- Gestión de estado asíncrono